

Project Update: CSSE230 Assignment Logger

1) Summary

The goal of the research project is to provide an avenue for CSSE230 students to reflect on an independent analysis of their performance on homework assignments. The logger is designed to perform just the data collection step of the process by recording data during the JUnit tests written for each assignment, including the related code diffs (which are redacted for privacy). The requirements include privacy by design, invisibility at runtime, compatibility with all student systems, and ease of integration into projects. More detail on the requirements and implementation are provided below.

The logger has been tested to validate its functionality and performance. A combinatorial testing approach was taken to account for potential interaction between features (e.g., resilience to runtime errors and new file creation) where plausible. It has been stress tested on real assignments to validate insignificant impact on disk space and user experience, and an analysis of the performance is provided later on. The specific logging time and space limits can be tuned by changing constants in the code.

2) Current Scope & Goals

- Scope now
 - Logger proof of concept only
- Out of scope
 - Analytics, dashboard, and email/SMS tools

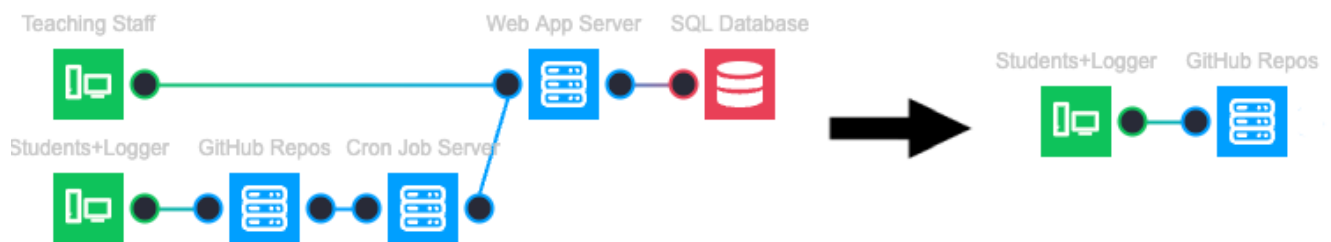


Figure 1: Scope change for initial trials.

- Goals
 - Validate technical reliability and privacy with dummy files
 - Confirm ease of integration into the course
 - Collect test and diff data
 - Iterate on instructor feedback

- Next steps
 - Running the GitHub Classroom pilot
 - Obtaining IRB approval
 - Trial on 1-2 assignments if it works as planned in pilot
-

3) Design

The system meets the following requirements:

- Drop-in compatibility
 - Java 15+
 - Cross-OS compatibility
 - Minimal refactoring for JUnit 5
 - Privacy by design
 - Uses dummy files (redacts code)
 - Easy opt-out
 - Invisible at runtime
 - Single-file Git diffs (run.tar)
 - Resilient to internal defects
 - Fails silently
 - Doesn't affect user experience
 - Max logger-related execution time: 500 ms
 - Max non-blocking (after tests) saving time: 3000 ms
 - File size bounds
 - Log tar file max size: 2 MB
 - Max diffed file size: 1 MB
 - Max overall repo files size: 10 MB
 - Self-shutoff in case of excess time/space consumption
 - Breaking any space limits
 - Max 2 of last 3 runs or 1 run exceeds 3 times the time limits
-

4) Implementation

The testSupport package ([latest version](#)) is composed of two classes: a singleton used for logging each test run (LoggingSingleton) and a class with JUnit lifecycle callbacks (LoggingExtension).

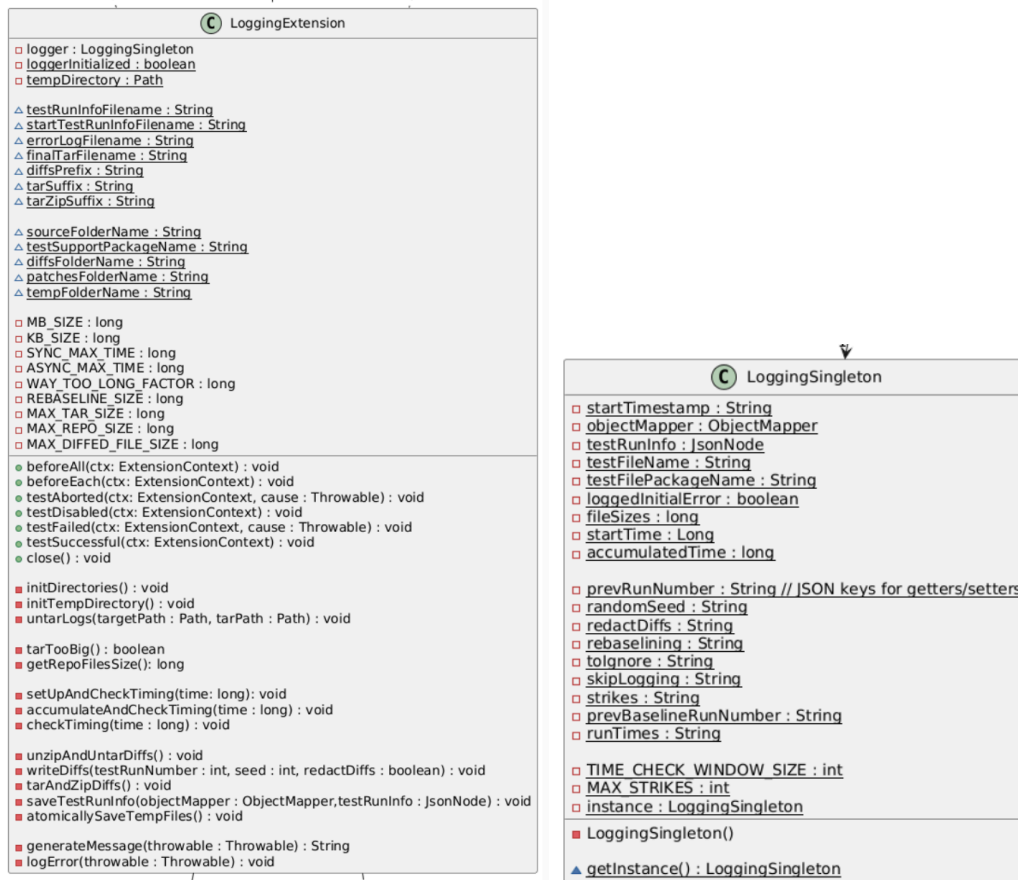


Figure 2: Diagrams for LoggingExtension and LoggingSingleton ([Full UML Design Link](#)).

As shown in Fig. 2, LoggingSingleton is accessed only through the callbacks, which are the only public methods (or fields) across both classes. Both are instantiated when tests are run. Each callback is used to record data on the tests and contains the following boilerplate (run in this order):

1. A surrounding try-catch that prevents any Throwable (i.e., an Error or Exception) from escaping the method
 - a. Any caught Throwable is logged to error-logs.txt (within another try-catch) and the logger is deactivated for all subsequent runs
 - b. Throws outside work normally
2. Checking for whether logging has been deactivated or not, skipping if so
3. Starting logger timing/checking time elapsed against the limit
4. At the end of the method, pausing logger timing/rechecking time elapsed against the limit

Additionally, neither recursion nor infinite loops are used.

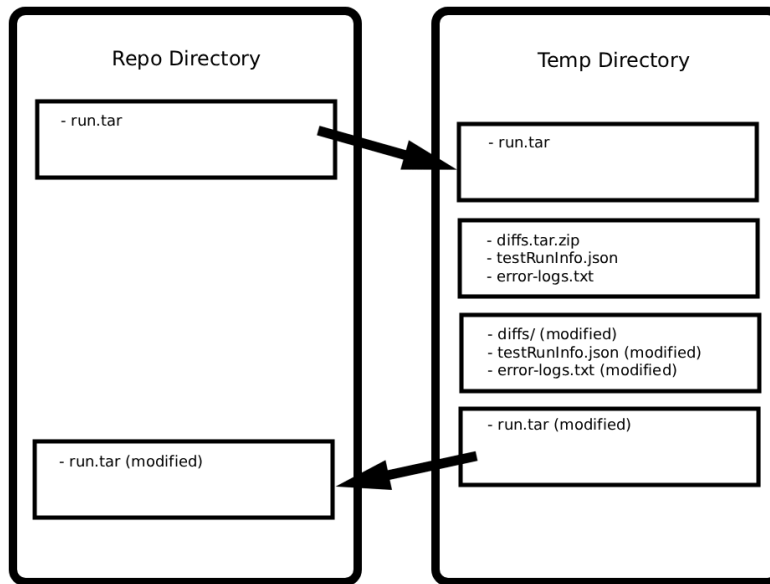


Figure 3: A diagram showing the movement of files between the testSupport package and temporary directory.

The first callback is `beforeAll`. It runs before all tests begin. If the size of all files in the repo is less than 10 MB and `run.tar` less than 2 MB, it initializes the logger singleton and untars `run.tar` from the testSupport package into a temporary directory (Fig. 3) to retrieve configurations from the previous `testRunInfo.json`. A temporary directory is used to avoid the possibility of intermediate files appearing in the repo in case of a crash. If `run.tar` is deleted, `beforeAll` and the other callbacks won't be run, allowing students to opt out easily.

Next are the callbacks that are run for each test: `beforeEach` before each test and `testSuccessful`, `testFailed`, `testAborted`, and `testDisable` after each one. These all share the same purpose of initializing/updating the status of a given test run to reflect how it went by updating the singleton. The time limit for all callbacks up to this point is 500 ms, and this is the end of the portion of the logging that blocks tests while it runs.

Finally, the `close()` method is called. Instead of a JUnit lifecycle callback, it is a shutdown hook to the current Java runtime. This makes it run after JUnit is finished, without affecting test run time. Therefore, it is non-blocking to the tests. Since this handles the diffing process for many files, its time limit is 3000 ms (although it stayed under a second in testing). The following steps (see the diagram above) are used in updating `diffs.tar.zip`:

1. Unzipping and untarring `diffs.tar.zip`
2. Updating diffs
 - a. Either creating a baseline (if none exists) or adding patches (diffs) for the code
 - i. Only .java files are diffed
 - ii. Individual files are limited to 1 MB meaning at most the uncompressed diffs for an individual file will be 2 MB (1 MB each for deletions and additions)
 - iii. Combined with the 10 MB overall limit for the repo, in the worst case, the most written is 20 MB (which would be millions of lines of code) in the

- temp directory, before compression. Even so, if that exceeds the timing threshold during or after running the tests, the logger is deactivated
- If the flag to redact the diffs is true, they are saved via a randomized (currently seeded with the first run's time) character mapping used to lossily redact all alphanumeric characters (see Fig. 4 below). These dummy files hide the meaning of the code while preserving its shape for diffing validation.
3. Tarring and zipping diffs.tar.zip

As shown in Fig. 3, the three log files (the updated diffs.tar.zip, testRunInfo.json, and error-logs.txt) are tarred back into run.tar and then moved from the temporary directory (using an atomic file move, to avoid log corruption in case of a crash) back into the testSupport package. The temp directory is then deleted (also cleared if an error occurs).

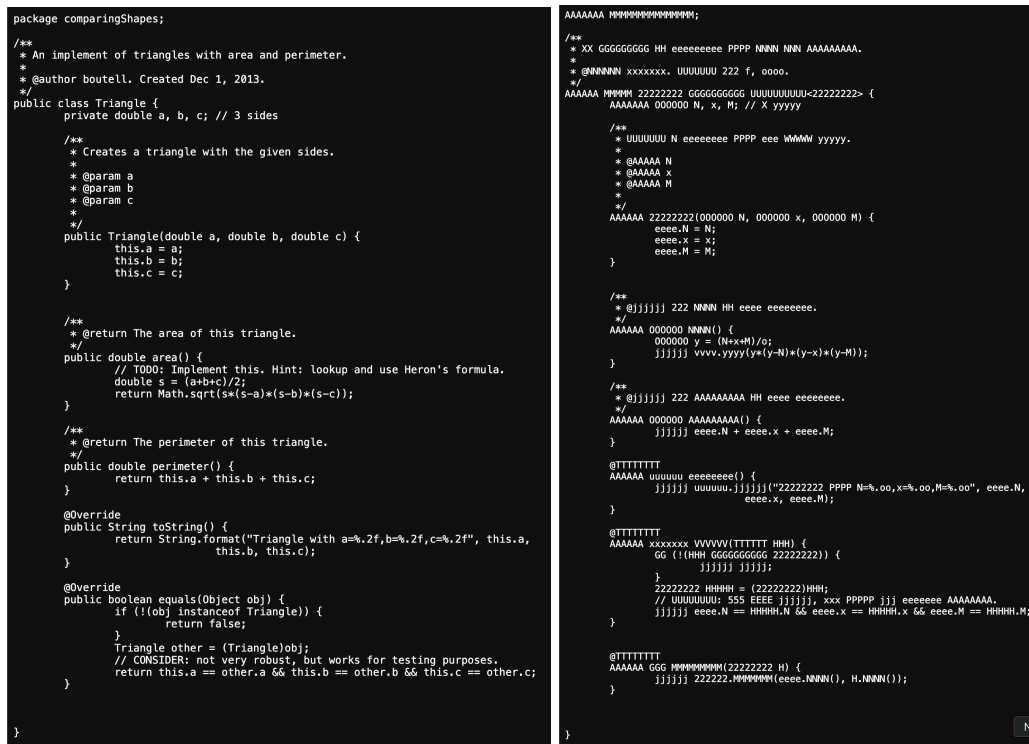


Figure 4: Both images show the same class, with the one on the right having been randomly redacted.

Note:

For performance purposes, there are actually multiple `diffs.tar.zip` files. As shown in Fig. 5, a new one (along with a new set of baselines) is created each time the prior one's uncompressed size (specifically of its `diffs`) exceeds 10 KB.

run copy				+
Name	^	Date Modified	Size	Kind
diffs_0_.tar.zip		Aug 12, 2025 at 6:53 AM	13 KB	ZIP archive
diffs_10_.tar.zip		Aug 12, 2025 at 6:58 AM	14 KB	ZIP archive
diffs_17_.tar.zip		Aug 12, 2025 at 7:03 AM	13 KB	ZIP archive
diffs_25_.tar.zip		Aug 12, 2025 at 7:05 AM	13 KB	ZIP archive
diffs_29_.tar.zip		Aug 12, 2025 at 7:15 AM	14 KB	ZIP archive
diffs_38_.tar.zip		Aug 12, 2025 at 7:19 AM	13 KB	ZIP archive
diffs_44_.tar.zip		Aug 12, 2025 at 7:22 AM	14 KB	ZIP archive
diffs_52_.tar.zip		Aug 12, 2025 at 7:25 AM	15 KB	ZIP archive
diffs_56_.tar.zip		Aug 12, 2025 at 7:31 AM	12 KB	ZIP archive
diffs_61_.tar.zip		Aug 12, 2025 at 7:32 AM	14 KB	ZIP archive
diffs_67_.tar.zip		Aug 12, 2025 at 7:34 AM	14 KB	ZIP archive
diffs_74_.tar.zip		Aug 12, 2025 at 7:39 AM	13 KB	ZIP archive
diffs_82_.tar.zip		Aug 12, 2025 at 7:41 AM	15 KB	ZIP archive
diffs_89_.tar.zip		Aug 12, 2025 at 7:43 AM	15 KB	ZIP archive
diffs_96_.tar.zip		Aug 12, 2025 at 7:46 AM	15 KB	ZIP archive
diffs_102_.tar.zip		Aug 12, 2025 at 7:49 AM	11 KB	ZIP archive
diffs_109_.tar.zip		Aug 12, 2025 at 7:49 AM	11 KB	ZIP archive
error-logs.txt		Aug 12, 2025 at 6:50 AM	Zero bytes	Plain Text
testRunInfo.json		Aug 12, 2025 at 7:49 AM	323 KB	JSON Document

Figure 5: A series of diffs.tar.zip files created during stress testing with over 100 runs.

This is referred to below as re-baselining. It lowers execution time by preventing diffs from growing quadratically, since otherwise files are repeatedly diffed against the same baseline with each run. Previous diffs.tar.zip are left compressed when untarred since their baselines aren't needed. They are then tarred, along with the other log files, and moved back into the repo. A more detailed schema for each of the files is linked below.

- Helpful links:
 - Log File Schema: [Link](#)
 - Logger Setup Steps: [Link](#)
 - Development Repo Link: [Link](#)

5) Validation & Results

Various tests have been conducted to make sure that the logger functions as expected, in both functionality and performance. Detailed test plans are recorded in the branches of this repository: [WarmupAndStretchingTesting](#).

Functionality Tests

Single Factor Tests

- Cross-OS testing (Windows/macOS)
 - Passed
- Java versions (15, 17, 21, 24)
 - Passed
- Multiple assignments (WarmupAndStretching, BinarySearchTree)
 - [BinarySearchTreeWithCompression](#)

- Passed
- Time limit sliding window shutdown (2 of last 3 exceed time limit)
 - Passed

Multiple Factor Tests

- Factors
 - A: Run order/history
 - B: Test volume pattern
 - C: File edit patterns
 - D: File churn type
 - E: Failure source pattern
- For factors that could interact, such as the creation of new files and runtime exceptions, a pairwise test was created
 - See the test plans (e.g., Test A-B-1), resulting [run.tar](#) files, and READMEs for more detail: [WarmupAndStretchingTesting/testing-A-B-1](#)

	A	B	C	D	E
A		X			X
B			X		X
C				X	X
D					X
E					

Figure 6: Interacting factors that needed to be tested are marked with an X

Performance Tests

Baseline Test

- See description here: [WarmupAndStretchingTesting/testing-performance-realistic-1](#)

Stress Test

- See description here: [WarmupAndStretchingTesting/testing-performance-stress-test-1](#)
- Environment
 - Hardware
 - Chip: Apple M1 Pro
 - Memory: 32 GB
 - Java Runtime
 - Java 15, for worst case performance
- Data (also in repositories)
 -  stressPerformanceData
- Metrics
 - 95th percentile (total) logging time: **613.6 ms**
 - This includes the shorter test run-blocking segment which adds execution time to the tests and the longer non-blocking diffing and saving
 - Log size after >100 runs: **554 KB**

- Crashes/Errors: 0
- Interpretation

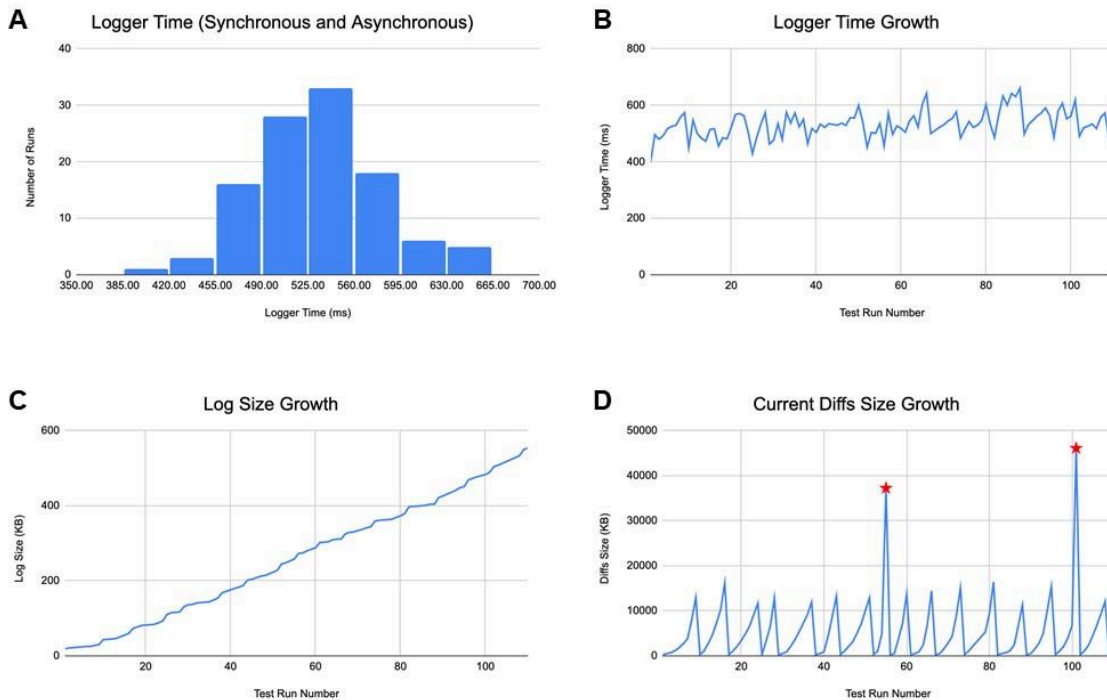


Figure 7: (A) Distribution of total logging times (blocking and non-blocking). (B) Total logging times over time. (C) Log size over time. (D) Size of the current set of diffs over time. The red stars are the largest two individual changes between runs (>10 KB), both reverting all prior changes.

In 95% of test runs, the logger added less than 615 ms. The majority of that time occurred after tests were already finished running. Evidence for this is that the logger never shut off; 2 of the last 3 runs never added 500 ms or more of run time over 100+ runs. The slowest overall time was 660 ms.

Despite the varied sizes of the changes in each test run, the log size (Fig. 7C) grew roughly linearly and the logging time (Fig. 7B) stayed relatively constant. This is because re-baselining was triggered when a set of diffs exceeds the 10 KB threshold (Fig. 7D), minimizing the main bottleneck of quadratically growing diff sizes.

The logger was able to meet all functionality and performance criteria outlined. However, to ensure that the results hold for every student's machine, further validation through a pilot is required.